



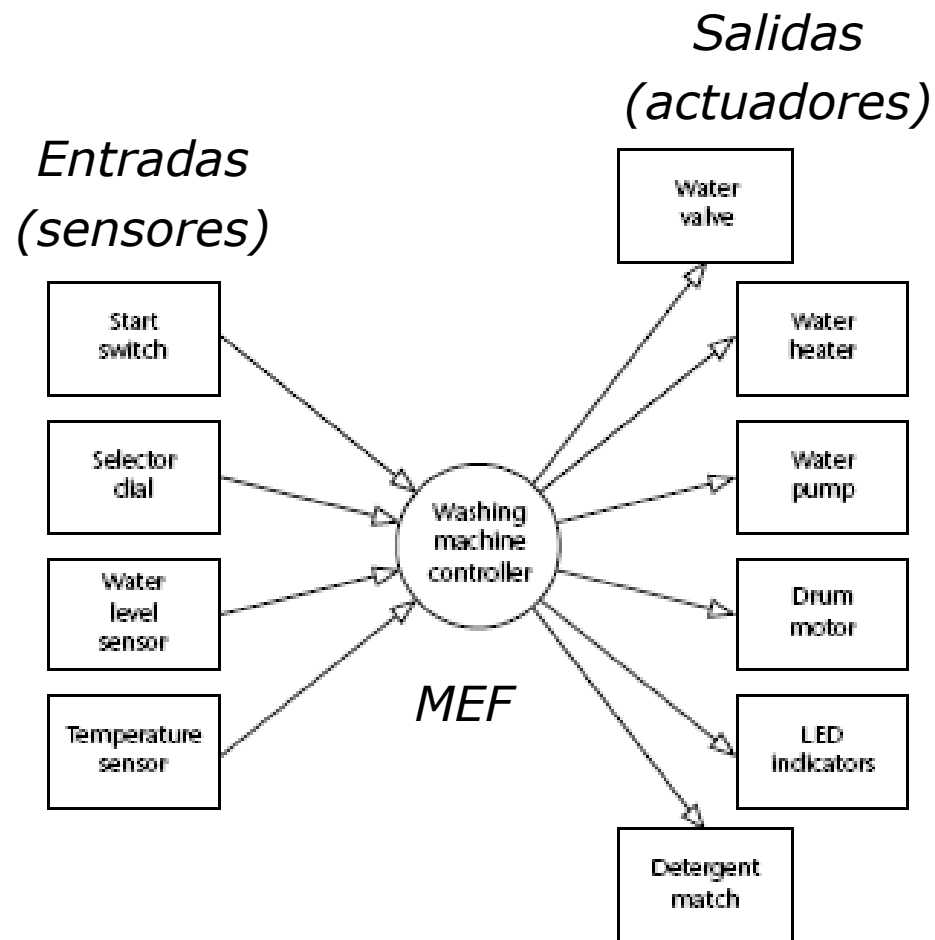
CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025

Facultad de Ingeniería
UNLP

Ejemplos de abstracción MEF
y su implementación en Software

Ing. José Juárez

Control de lavadora



Control de lavadora

Modelo Conceptual:

1. El usuario selecciona el programa de lavado
2. El usuario presiona START
3. Se activa la traba de seguridad
4. Se abre la válvula de agua
5. Si el programa de lavado requiere jabón, se abre la compuerta de jabón y luego se cierra
6. La válvula de agua se cierra cuando se alcanza el nivel
7. Si el programa requiere agua caliente, se encienden los calentadores hasta que se alcance la temperatura especificada
8. Se enciende el motor y se comienza la secuencia de movimientos adecuadas para el programa elegido
9. Al finalizar, la bomba de desagote se enciende para vaciar el tambor y se apaga cuando se detecte que este está vacío
10. Se desactiva la traba de seguridad

Control de lavadora

Posibles Estados:

INIT, START, FILL_DRUM, HEAT_WATER, WASH_01, WASH_02, ..., PUMP, ERROR

Funciones a implementar para las entradas:

Read_Selector_Dial()	//Leer selector de programa
Read_Start_Switch()	//leer pulsador de arranque
Read_Water_Level()	//leer sensor de nivel de agua
Read_Water_Temperature()	//leer sensor de temperatura

Las funciones de verificación de eventos de entrada, devuelven 0 si no hay evento y 1 si lo hay evento. Son No bloqueantes, sin retardos ni condiciones de espera.

Funciones a implementar para las salidas:

Control_Detergent_Hatch()	//controlar dispenser de jabon
Control_Door_Lock()	//controlar traba de seguridad
Control_Motor()	//accionar motor
Control_Pump()	//controlar bomba de desagote
Control_Water_Heater()	//controlar calentadores de agua
Control_Water_Valve()	//controlar válvula de agua

Control de lavadora

```
/*-----*/
```

Washer.C

```
/*-----*/
```

```
#include "Washer.h"
```

```
//----- Private data type declarations -----
```

```
typedef enum {INIT, START, FILL_DRUM,  
             HEAT_WATER, WASH_01, WASH_02, ERROR}  
eSystem_state;
```

```
//----- Private function prototypes -----
```

```
char WASHER_Read_Selector_Dial(void);  
char WASHER_Read_Start_Switch(void);  
char WASHER_Read_Water_Level(void);  
char WASHER_Read_Water_Temperature(void);  
void WASHER_Control_Detergent_Hatch(bit);  
void WASHER_Control_Door_Lock(bit);  
void WASHER_Control_Motor(bit);  
void WASHER_Control_Pump(bit);  
void WASHER_Control_Water_Heater(bit);  
void WASHER_Control_Water_Valve(bit);
```

```
//----- Private constants -----
```

```
#define OFF 0
```

```
#define ON 1
```

```
#define MAX_FILL_DURATION          1000L
```

```
#define MAX_WATER_HEAT_DURATION 1000L
```

```
#define WASH_01_DURATION          30000L
```

```
//----- Private variables -----
```

```
static eSystem_state System_state;
```

```
static long Time_in_state;
```

```
static char Program;
```

```
// 10 programas diferentes
```

```
// cada uno con uso o no de detergente
```

```
static char Detergent[10] = {1,1,1,0,0,1,0,1,1,0};
```

```
// cada uno usa o no agua caliente
```

```
static char Hot_Water[10] = {1,1,1,0,0,1,0,1,1,0};
```

```
/*-----*/
```

```
void WASHER_Init(void)
```

```
{
```

```
    System_state = INIT;
```

```
}
```

```
/*-----*/
```

Control de lavadora: MEF

```
void WASHER_Update(void)
{
    switch (System_state)
    {
        case INIT:
            //código ...
            break;
        case START:
            //código
            break;
        case FILL_DRUM:
            //código
            break;
        case HEAT_WATER:
            //código
            break;
        case WASH_01:
            //código
            break;
        //idem WASH_02: ... etc
        case ERROR:
            //código
            break;
    }
}
```

Actualizaremos la MEF con:

void WASHER_Update(void)

Esta función de actualización se implementará mediante sentencias switch-case con el formato visto en clase.

Control de lavadora

- Para temporizar la ejecución de la MEF podríamos hacer:

```
void main(void)
{
    // Inicializar board y tareas

    WASHER_Init();

    while(1) {                // Super Loop

        WASHER_Update();

        _delay_ms(1000);
    }
}
```

- Pero vamos a utilizar una metodología más eficiente que profundizaremos en las próximas clases:

Usaremos interrupción periódica de Timer para temporizar la actualización de la MEF

Control de lavadora

```
/* ----- */
void main(void)
{
    // Inicializar MCU, LCD, tareas, etc

    while (1) { // Super Loop

        if (Flag_MEF) {

            WASHER_Update();

            Flag_MEF = 0;

        }

    }
} /* ----- */
```

```
// Variables globales
volatile uint8_t Flag_MEF=0;
volatile uint8_t cont_MEF=0;
```

Mejoraremos la modularización en el próximo ejemplo

```
// MEF se ejecuta para el ejemplo 1 vez por segundo

/* ----- */
ISR TIMER : ocurre cada 1 ms
/* ----- */
ISR (TIMER0_CompA_vect)
{
    if (++cont_MEF==1000) {

        Flag_MEF=1;           //ejecutar cada 1s
        cont_MEF=0;

    }

}
```


Control de lavadora: caso por caso

case INIT:

// Motor off

WASHER_Control_Motor(OFF);

// Pump off

WASHER_Control_Pump(OFF);

// Heater off

WASHER_Control_Water_Heater(OFF);

// Valve closed

WASHER_Control_Water_Valve(OFF);

// Esperar hasta que se presione START

If (WASHER_Read_Start_Switch() == 1)
{

// Start presionado...

// Leer selector dial

Program = WASHER_Read_Selector_Dial();

// cambiar estado

System_state= START;

}

break;

Actualizar salidas

*Lee entrada
asociada al
estado *.*

Estado siguiente

*Ejemplo: lectura del pulsador
(No bloqueante)*

```
char WASHER_Read_Start_Switch(void)  
{  
    //polling periódico  
    if (pulsador Start presionado)  
    {  
        // puede tener doble verificación  
        // y algoritmo anti-rebote  
        return 1;  
    }  
    else  
    {  
        return 0;  
    }  
}
```

Control de lavadora: caso por caso

case START:

```
// Trabar puerta
WASHER_Control_Door_Lock(ON);
// abrir válvula agua
WASHER_Control_Water_Valve(ON);
// abrir válvula de detergente (si corresponde)
if (Detergent [Program] == 1)
{
    WASHER_Control_Detergent_Hatch(ON);
}

// ir al siguiente estado
System_state = FILL_DRUM;
Time_in_state = 0;
break;
```

case FILL_DRUM:

```
// esperar hasta que se llene el tanque
if (++Time_in_state >= MAX_FILL_DURATION)
    // incluye Timeout
    System_state= ERROR; // hay problemas...
if (WASHER_Read_Water_Level() == 1)
{ // tanque lleno
    // se requiere agua caliente?
    if (Hot_Water [Program] == 1)
    {
        WASHER_Control_Water_Heater(ON);
        // ir a siguiente estado
        System_state= HEAT_WATER;
        Time_in_state = 0;
    } else {
        // caso agua fría - comenzar programa
        System_state = WASH_01;
        Time_in_state = 0;
    }
}
break;
```

Control de lavadora: caso por caso

case HEAT_WATER:

// esperar se caliente el agua

if (++Time_in_state >= MAX_WATER_HEAT_DURATION)

{

// Timeout...

System_state= ERROR;

}

// verificar temperatura del agua

if (WASHER_Read_Water_Temperature() == 1)

{

// agua a temperatura deseada

// próx estado

System_state = WASH_01;

Time_in_state = 0;

}

break;

case WASH_01:

// WASH_01

WASHER_Control_Motor(ON);

if (++Time_in_state >= WASH_01_DURATION)

{

System_state = WASH_02;

Time_in_state = 0;

}

break;

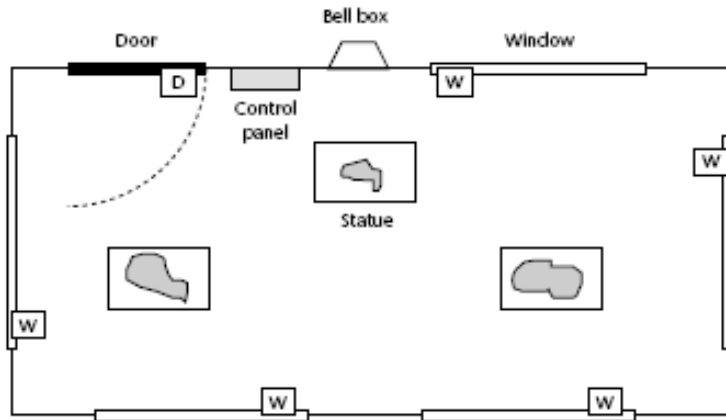
//demás estados omitidos

case ERROR:

// activar alarma

break;

Implementación de un alarma domiciliaria



Las ventanas y las puertas tienen sensores magnéticos. En el exterior se encuentra la sirena.



En el interior se encuentra El teclado, el display y un buzzer

El usuario arma la alarma colocando la clave de 4 dígitos, y cuenta con un tiempo de 60 seg para salir.

De la misma manera al entrar tiene 60 seg para desactivarla colocando la password, de lo contrario la alarma sonará

Modelado y especificación: MEF

- Inicialmente el sistema esta en el estado *Desarmado*, hasta que se ingrese la password correcta.
- A partir de aquí entramos al modo *Armando* que espera 60 seg.
- Luego el sistema esta *Armado* monitoreando los sensores. Si un sensor de ventana se activa, se pasa al modo *Intruso*, en cambio si el sensor de puerta se activa se pasa a modo *Desarmando*.
- En el estado *Desarmando* se espera 60seg para que el usuario autorizado pueda ingresar la clave. Si se ingresa la clave correcta el sistema para a *Desarmado* Caso contrario pasa a estado *Intruso*.
- En el estado *Intruso* la alarma suena indefinidamente hasta que se ingrese la clave correcta.
- Con esta información definimos el “diagrama de estados”

Tarea: hacer el diagrama de estados completo

Arquitectura del Software

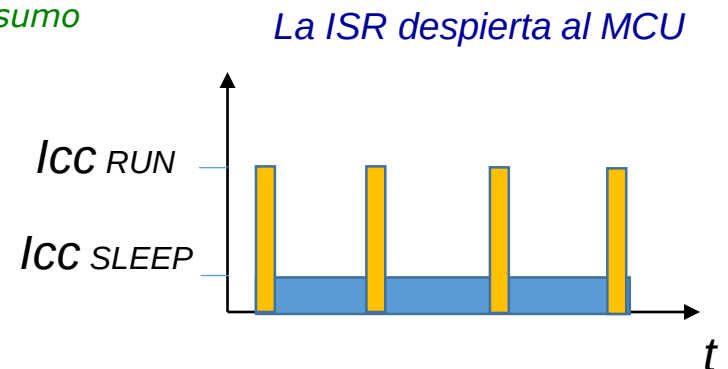
- **Temporización de Tareas:**
 - La temporización será por Interrupción periódica de Timer (Tick del sistema)
 - La interrupción de Timer será cada 10 ms
 - La maquina de estados se actualizará cada 100ms
 - Se utilizarán Funciones no bloqueantes para el manejo de entradas-salidas
- **Módulos del software:**
 - Main.c, main.h
 - Inicialización del sistema y super loop
 - sEOS.c, sEOS.h (una mejora en la modularización respecto al ejemplo anterior)
 - Planificador y Despachador cooperativo multitarea
 - Intruder.c, Intruder.h
 - Implementación MEF de la alarma
 - Keypad.c, Keypad.h
 - Biblioteca de funciones para leer el teclado
 - Lcd.c, Lcd.h
 - Biblioteca de funciones para manejo de LCD

Implementación: main()

```
void main(void)
{
    MCU_Init();
    // Inicializar LCD
    LCD_Init();
    // inicializar teclado
    KEYPAD_Init();
    // inicializar maq de estados y buffers
    INTRUDER_Init();
    // configurar timer (10ms tick)
    sEOS_Init_Timer(10);

    while(1)
    {
        sEOS_Dispatch_Tasks(); // ejecutar tareas
        sEOS_Go_To_Sleep(); // modo idle bajo consumo
    }
}
```

```
void INTRUDER_Init(void)
{
    System_state = DISARMED;
    LCD_Write_String("\nDisarmed");
    // Alarma apagada
    Sounder_pin = 0;
    State_call_count = 0;
}
```



Implementación: planificación y despacho

// ahora las variables globales son privadas

static volatile uint8_t MEF_flag=0;

static volatile uint8_t cont_MEF=0;

*La función sEOS_Init_Timer(10)
Inicializa el timer0 por ejemplo para
que interrumpa periódicamente
cada 10ms.*

*En la ISR se planifica que tarea
ejecutar de acuerdo a su
temporización.*

*La función sEOS_Dispatch_Tasks()
se ejecuta luego de cada ISR ()
para despachar secuencialmente
las tareas de acuerdo a lo
planificado.*

ISR (Timer0_CompA_vect) // cada 10 ms
{

// actualizar MEF cada 100 ms

if (++cont_MEF==10) {

MEF_flag=1;

cont_MEF=0;

}

*// podría haber otras tareas a
temporizar....*

}

void sEOS_Dispatch_Tasks (void) {

// actualizar MEF cada 100 ms

if (MEF_flag) {

MEF_flag = 0;

INTRUDER_Update(); // MEF

}

*// podría haber otras tareas a
despachar....*

}

Implementación: MEF

```
void INTRUDER_Update(void) // llamada cada 100 ms
```

Si el modelo es MOORE

```
{
```

```
    // Contar numero de interrupciones
```

```
    State_call_count++;
```

```
    switch (System_state)
```

```
    {
```

```
        case DISARMED:
```

```
            if (New_state) // si se cambia de estado hay que informarlo en el LCD (salida)
```

```
            {
```

```
                LCD_Write_String ("\nDisarmed");
```

```
                New_state = 0;
```

```
            }
```

```
            // Alarma apagada
```

```
            Sounder_pin = 0;
```

```
            // esperar por clave correcta ...
```

```
            if (INTRUDER_Get_Password() == 1)
```

```
            {
```

```
                System_state = ARMING; //pasar a armando...
```

```
                New_state = 1;
```

```
                State_call_count = 0;
```

```
                break;
```

```
            }
```

```
        break;
```

Actualiza
Salidas
del estado

Lee entrada

Estado siguiente

Condición de entrada
al estado. Única vez
(entry condition)

Implementación: MEF

Si el modelo es MEALY

```
void INTRUDER_Update(void)
{
    // Contar numero de interrupciones para calcular el tiempo en cada estado
    State_call_count++;

    switch (System_state)
    {
        case DISARMED:
            // esperar por clave correcta ...
            if (INTRUDER_Get_Password() == 1)           Leer entrada relevante para este estado
            {
                System_state = ARMING; //pasar a armando...   Siguiente estado
                LCD_Write_String("\nArming...");
                State_call_count = 0;                       Actualiza salidas (Mealy)
                break;
            }
        break;
    }
    Notar que si no hay transición de estado no hay cambios en las salidas
}
```

Implementación: MEF

case **ARMING**:

```
// Esperar 60 segundos (600 ticks)  
if (++State_call_count > 600)  
{  
    System_state = ARMED;  
    LCD_Write_String("\\nArmed");  
    State_call_count = 0;  
  
}  
break;
```

Implementación: MEF

case *ARMED*:

```
if (INTRUDER_Check_Window_Sensors() == 1) // Chequear sensores de ventanas
{
    // Intruso detectado
    System_state = INTRUDER;
    LCD_Write_String("\n** INTRUDER! **");
    //Activar alarma
    Sounder_pin = 1;
    State_call_count = 0;
}
if (INTRUDER_Check_Door_Sensor() == 1) // chequear sensor de puerta
{
    System_state = DISARMING; // Puede ser un usuario autorizado
    LCD_Write_String("\nDisarming...");
    State_call_count = 0;
}
if (INTRUDER_Get_Password() == 1) // Si hay clave correcta desarmar
{
    System_state = DISARMED;
    LCD_Write_String("\nDisarmed");
    // Alarma apagada
    Sounder_pin = 0;
    State_call_count = 0;
}
```

break;

*Notar que cada entrada se evalúa por separado.
Podría implementarse un switch-case dentro del estado
para evaluar los eventos de entrada en lugar de if's*

Implementación: MEF

case *DISARMING*:

// Esperar 60 seg para que ingrese la clave correcta, sino ALARMA

if (++State_call_count > 1200)

{

System_state = INTRUDER;

LCD_Write_String("\n** INTRUDER! **");

//Activar alarma

Sounder_pin = 1;

State_call_count = 0;

break;

}

// mientras tanto chequear sensores de ventana

if (INTRUDER_Check_Window_Sensors() == 1)

{

// intruso detectado

System_state = INTRUDER;

LCD_Write_String("\n** INTRUDER! **");

//Activar alarma

Sounder_pin = 1;

State_call_count = 0;

break;

}

if (INTRUDER_Get_Password() == 1) *// si hay clave correcta desarmar*

{

System_state = DISARMED;

LCD_Write_String("\nDisarmed");

// Alarma apagada

Sounder_pin = 0;

State_call_count = 0;

break;

Implementación: MEF

```
case INTRUDER:
```

```
    // Esperar sonando hasta que se ingrese la clave correcta
```

```
    if (INTRUDER_Get_Password() == 1)
```

```
    {
```

```
        System_state = DISARMED;
```

```
        LCD_Write_String("\nDisarmed");
```

```
        // Alarma apagada
```

```
        Sounder_pin = 0;
```

```
        State_call_count = 0;
```

```
    }
```

```
break;
```

Implementación: funciones de verificación

Las funciones de verificación de eventos de entrada, devuelven 0 si no hay evento y 1 si lo hay. Son No bloqueantes, sin retardos ni condiciones de espera.

```
char INTRUDER_Check_Window_Sensors(void)
{
    // un solo sensor en el ejemplo
    if (Window_sensor_pin == 0)
    {
        // Intruso detectado...
        return 1;
    }
    // Default
    return 0;
}
```

```
char INTRUDER_Check_Door_Sensor(void)
{
    // Sensor de puerta
    if (Door_sensor_pin == 0)
    {
        // Puerta abierta
        return 1;
    }
    // Default
    return 0;
}
```

Implementación: funciones de verificación

```
char INTRUDER_Get_Password(void)
{
    // Hay nuevo dato en el buffer de teclado?
    if (KEYPAD_scan(&Key) == 0) No bloqueante, sin retardos ni condiciones de espera.
    {
        return 0; // No Hay dato nuevo y por lo tanto no hay clave correcta
    }
    // Se ha apretado una tecla, se espera 50seg mas para leer la clave completa
    if (State_call_count > 500)
    {
        // timeout, no se completo el ingreso de la clave, reiniciar
        State_call_count = 0;
        Tecla=0;
        return 0;
    }
    //Escribo y guardo tecla presionada
    LCD_Write_Char(Key);
    Input [Tecla++] = Key;
    // tengo las 4 teclas?
    if(Tecla==4) {
        Tecla=0;
        return validar_clave(); //si la clave es correcta retorna 1, incorrecta 0
    } else
        return 0; //no tengo las cuatro teclas aún
}
```


Bibliografía

Los ejemplos fueron extraídos de la siguiente bibliografía:

- *“Embedded Microcomputer System, Real-Time Interfacing”*. J. Valvano 2nd Ed. Thomson 2007.
- *“Patterns for time-triggered Embedded Systems”*, Michael J. Pont. 2014
Published by SafeTTy Systems (pdf disponible en la web)